CSC258 Distributed Systems
Instructor: Dr. Abeer Abdel Khaleq
Spring 2026
Tam Vo

Implementation Assignment #2: Docker and containers

1. Describe the environment you used and programming language.

2. Describe the Docker containers you developed and Docker files.

3. Describe how you ran your containers and the results you got.

4. Instructions on how to build and run the application with the port number used.

5. Reflection on testing, observations, challenges, etc.

6. References as applicable for any resources you used.

Introduction (#1-4)

The environment that I used was Java 15+ with the package manager being Apache Maven. Using Docker, I created a compose.yml file that detailed the dependency order of the project. Alongside this I created Dockerfile.client and Dockerfile.server to have a custom startup with the client and server images respectively. With this after using ***docker compose up --build server*** command and the ***docker compose up --build --scale client=10 client*****,** I was able to deploy this simple server and client server.

The server successfully built in Java and hard-coded exposed port 8080 for the clients to connect.

```
○ tvo@tvos-MacBook-Pro-5 HW2 % docker compose up --build server

  [+] Building 1.9s (16/16) FINISHED
   => [internal] load local bake definitions                                    0.0s
   => => reading from stdin 559B                                                0.0s
   => [internal] load build definition from Dockerfile.server                   0.0s
   => => transferring dockerfile: 271B                                          0.0s
   => resolve image config for docker-image://docker.io/docker/dockerfile:1     0.9s
   => [auth] docker/dockerfile:pull token for registry-1.docker.io              0.0s
   => CACHED docker-image://docker.io/docker/dockerfile:1@sha256:b6afd42430b1   0.0s
   => [internal] load metadata for docker.io/library/maven:latest               0.6s
   => [auth] library/maven:pull token for registry-1.docker.io                  0.0s
   => [internal] load .dockerignore                                             0.0s
   => => transferring context: 2B                                               0.0s
   => [1/5] FROM docker.io/library/maven:latest@sha256:4f82a03a7d6679281952d6   0.0s
   => [internal] load build context                                             0.0s
   => => transferring context: 3.48kB                                           0.0s
   => CACHED [2/5] WORKDIR /src                                                 0.0s
   => CACHED [3/5] COPY pom.xml pom.xml                                         0.0s
   => CACHED [4/5] RUN mvn clean install                                        0.0s
   => [5/5] COPY src/main/java/HW2 ./src/main/java/HW2                          0.0s
   => exporting to image                                                        0.0s
   => => exporting layers                                                       0.0s
   => => writing image sha256:c41ad16535108a57359e5bc2fa04fe07eec64534cb72b87   0.0s
   => => naming to docker.io/library/hw2-server                                 0.0s
   => resolving provenance for metadata file                                    0.0s
  [+] up 3/3
   ✔ Image hw2-server       Built                                               1.9s
   ✔ Network hw2_default    Created                                             0.0s
   ✔ Container hw2-server-1 Created                                             0.0s
  Attaching to server-1
```

The client successfully built in Java to connect to the exposed port 8080.

```
tvo@tvos-MacBook-Pro-5 HW2 % docker compose up --build --scale client=10 client
 => => transferring dockerfile: 259B                                                    0.0s
 => [server internal] load build definition from Dockerfile.server                      0.0s
 => => transferring dockerfile: 271B                                                    0.0s
 => [server] resolve image config for docker-image://docker.io/docker/dockerfile:1      0.5s
 => CACHED [client] docker-image://docker.io/docker/dockerfile:1@sha256:b6afd42430b15f2d2a4c5a0  0.0s
 => [server internal] load metadata for docker.io/library/maven:latest                  0.3s
 => [client internal] load .dockerignore                                                0.0s
 => => transferring context: 2B                                                         0.0s
 => [client 1/5] FROM docker.io/library/maven:latest@sha256:4f82a03a7d6679281952d628131299b1be8  0.0s
 => [client internal] load build context                                                0.0s
 => => transferring context: 295B                                                       0.0s
 => CACHED [server 2/5] WORKDIR /src                                                    0.0s
 => CACHED [server 3/5] COPY pom.xml pom.xml                                            0.0s
 => CACHED [server 4/5] RUN mvn clean install                                           0.0s
 => CACHED [server 5/5] COPY src/main/java/HW2 ./src/main/java/HW2                      0.0s
 => [client] exporting to image                                                         0.0s
 => => exporting layers                                                                 0.0s
 => => writing image sha256:dcd8e8ac4cea72f80303bb045efb77e006454a4617e472c473afd93788e533bf  0.0s
 => => naming to docker.io/library/hw2-client                                           0.0s
 => [server] exporting to image                                                         0.0s
 => => exporting layers                                                                 0.0s
 => => writing image sha256:c41ad16535108a57359e5bc2fa04fe07eec64534cb72b875a10c8d5e9e6fb8ae  0.0s
 => => naming to docker.io/library/hw2-server                                           0.0s
 => [server] resolving provenance for metadata file                                     0.0s
 => [client] resolving provenance for metadata file                                     0.0s
[+] up 13/13
 ✔ Image hw2-client        Built                                                        1.4s
 ✔ Image hw2-server        Built                                                        1.4s
 ✔ Container hw2-server-1  Running                                                      0.0s
 ✔ Container hw2-client-10 Created                                                      0.1s
 ✔ Container hw2-client-1  Created                                                      0.1s
 ✔ Container hw2-client-2  Created                                                      0.1s
 ✔ Container hw2-client-3  Created                                                      0.1s
 ✔ Container hw2-client-4  Created                                                      0.1s
 ✔ Container hw2-client-5  Created                                                      0.1s
 ✔ Container hw2-client-6  Created                                                      0.1s
 ✔ Container hw2-client-7  Created                                                      0.1s
 ✔ Container hw2-client-8  Created                                                      0.1s
 ✔ Container hw2-client-9  Created                                                      0.1s
Attaching to client-1, client-10, client-2, client-3, client-4, client-5, client-6, client-7, client-8
, client-9
```

Reflection of Testing, Observations and Challenges

Testing was done before the deployment of the Server and Client. I used Apache Maven to test the Client and Server Application by mocking the expected output with the modified input or function call using the Java Library Mocktio. Using **mvn clean test,** you can run the tests created for the Client and server. Mocking the client needed to be updated since there should be a flag to notate sleeping messages or not from the previous assignment.

From my observations, the docker containers seems to be installing the same packages to the client images. This is interesting since I each image should be the same, but why is it that they are having duplicated installs without copying from another image to another.

One of the biggest challenges of this project was learning the coordination of the docker containers, before I notated the dependencies in the yaml file. I would run the client first, but I would get errors, since the server's Ip address was not available yet.

Testing

```
tvo@tvos-MacBook-Pro-5 HW2 % mvn clean test
[INFO] Scanning for projects...
[INFO]
[INFO] ---------------< com.tamkhaivo:distributed-systems-hw1 >----------------
[INFO] Building distributed-systems-hw1 1.0-SNAPSHOT
[INFO]   from pom.xml
[INFO] --------------------------------[ jar ]---------------------------------
[INFO]
[INFO] --- clean:3.2.0:clean (default-clean) @ distributed-systems-hw1 ---
[INFO] Deleting /Users/tvo/Documents/portfolio/Distributed-Systems/HW2/target
[INFO]
[INFO] --- dependency:3.6.1:properties (default) @ distributed-systems-hw1 ---
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ distributed-systems-hw1 ---
[INFO] skip non existing resourceDirectory /Users/tvo/Documents/portfolio/Distributed-Systems/HW2/src/
main/resources
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ distributed-systems-hw1 ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 2 source files with javac [debug release 17] to target/classes
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ distributed-systems-hw1 ---
[INFO] Copying 1 resource from src/test/resources to target/test-classes
[INFO]
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ distributed-systems-hw1 ---
[INFO] Recompiling the module because of changed dependency.
[INFO] Compiling 1 source file with javac [debug release 17] to target/test-classes
[INFO]
[INFO] --- surefire:3.2.2:test (default-test) @ distributed-systems-hw1 ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running HW2.ClientServerTests
Sent: Client #1: Test Message 1
Received from Server: Hello from Server
Sent: Client #1: Test Message 2
Server disconnected
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.351 s -- in HW2.ClientServerT
ests
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
```

References

1. https://site.mockito.org
2. https://www.w3schools.com/java/java_threads.asp
3. https://docs.docker.com/compose/
4. https://www.geeksforgeeks.org/java/socket-programming-in-java/